LeetCode Practice

21 Medium: Binary Search

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

21
MEDIUM

Patterns (in order)

1. Binary Search (21)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

21 questions | tap a title to jump to it

Medium questions (21) Go >

Binary Search (21) Go >

M 1. Search in Rotated Sorted Array LC 33
M 2. Find First and Last Position of Element in Sorted Array LC 34
M 3. Search in Rotated Sorted Array II LC 81
M 4. Find Minimum in Rotated Sorted Array LC 153
M 5. Find Peak Element LC 162
M 6. Longest Increasing Subsequence LC 300
M 7. Design Hit Counter LC 362
M 8. Random Pick with Weight LC 528
M 9. My Calendar I LC 729
M 10. My Calendar II LC 731
M 11. Peak Index in a Mountain Array LC 852
M 12. Koko Eating Bananas LC 875
M 13. Time Based Key-Value Store LC 981
M 14. Capacity to Ship Packages Within D Days LC 1011
M 15. Longest Arithmetic Subsequence LC 1027
M 16. Missing Element in Sorted Array LC 1060
M 17. Longest Repeating Substring LC 1062
M 18. Snapshot Array LC 1146
M 19. Minimum Number of Days to Make m Bouquets LC 1482
M 20. Find the Index of the Large Integer LC 1533
M 21. Magnetic Force Between Two Balls LC 1552

Medium questions (21)

MEDIUM | TOPIC 1 OF 1

Binary Search

21 Medium questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Search in Rotated Sorted Array LC 33
2. Find First and Last Position of Element in Sorted Array LC 34
3. Search in Rotated Sorted Array II LC 81
4. Find Minimum in Rotated Sorted Array LC 153
5. Find Peak Element LC 162
6. Longest Increasing Subsequence LC 300
7. Design Hit Counter LC 362
8. Random Pick with Weight LC 528
9. My Calendar I LC 729
10. My Calendar II LC 731
11. Peak Index in a Mountain Array LC 852
12. Koko Eating Bananas LC 875
13. Time Based Key-Value Store LC 981
14. Capacity to Ship Packages Within D Days LC 1011
15. Longest Arithmetic Subsequence LC 1027
16. Missing Element in Sorted Array LC 1060
17. Longest Repeating Substring LC 1062
18. Snapshot Array LC 1146
19. Minimum Number of Days to Make m Bouquets LC 1482
20. Find the Index of the Large Integer LC 1533
21. Magnetic Force Between Two Balls LC 1552

MEDIUM | Binary Search | LeetCode #33 | 1 of 21 | 1/21

Search in Rotated Sorted Array

QUESTION

There is an integer array nums sorted in ascending order (with distinct values).

Prior to being passed to your function, nums is possibly left rotated at an unknown index k (1 <= k < nums.length) such that the resulting array is [nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]] (0-indexed). For example, [0,1,2,4,5,6,7] might be left rotated by 3 indices and become [4,5,6,7,0,1,2].

Given the array nums after the possible rotation and an integer target, return the index of target if it is in nums, or -1 if it is not in nums.

You must write an algorithm with O(log n) runtime complexity.

Example 1	
Input	`nums = [4,5,6,7,0,1,2], target = 0`
Output	`4`

Example 2	
Input	`nums = [4,5,6,7,0,1,2], target = 3`
Output	`-1`

Example 3	
Input	`nums = [1], target = 0`
Output	`-1`

CONSTRAINTS

- 1 <= nums.length <= 5000
- 10^4 <= nums[i] <= 10^4
- All values of nums are unique.
- nums is an ascending array that is possibly rotated.
- 10^4 <= target <= 10^4

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Array was sorted then rotated at some unknown pivot. Find target in O(log n). Right?

All values are distinct. Return -1 if not found."

"[4,5,6,7,0,1,2], target=0 -> 4 ok. target=3 -> -1 ok."

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    if nums[i] == target:
        return i
return -1
```

TIME	SPACE
O(n)	O(1)

WHY IT IS SLOW

Ignores the sorted-then-rotated structure. At each step one half is always sorted, so a modified binary search is O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Standard binary search — but at each step, identify which half is sorted.

If nums[left] <= nums[mid]: left half is sorted.

If target in [nums[left], nums[mid]): search left.

Else search right.

Else: right half is sorted.

If target in (nums[mid], nums[right]]: search right.

Else search left.

Time: O(log n). Space: O(1)."

Key insight: one half of a rotated sorted array is always normally sorted — identifying which half in O(1) lets standard binary search proceed on the sorted side.

PHASE 4 CLEAN CODE

```
class _33_SearchRotatedArray:
    def search(self, nums: List[int], target: int) -> int:
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = (left + right) // 2
            if nums[mid] == target:
                return mid
            if nums[left] <= nums[mid]:              # left
 half sorted
                if nums[left] <= target < nums[mid]:
                    right = mid - 1
                else:
                    left = mid + 1
            else:                                    # righ
t half sorted
                if nums[mid] < target <= nums[right]:
                    left = mid + 1
                else:
                    right = mid - 1
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace search([4,5,6,7,0,1,2], target=0)."

lo=0, hi=6. mid=3. nums[mid]=7.

Left half [4,5,6,7] sorted (nums[lo]=4 <= nums[mid]=7).

target=0: 0 in [4..7]? No -> search right: lo=4.

lo=4, hi=6. mid=5. nums[mid]=1. Left [0,1] sorted (nums[4]=0 <= nums[5]=1).

target=0: 0 in [0..1]? Yes -> search left: hi=4.

lo=4, hi=4. mid=4. nums[4]=0==target -> return 4 ok

target=3: never found -> return -1 ok

WHY check which half is sorted: pivot is always on the boundary of the two halves.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n)	**O(1)**

The key: at least one half is always sorted in a rotated array — use that half

to decide where to search. Checking nums[left] <= nums[mid] tells us which half.

Given more time I'd ask: what if there are duplicates?

That's #81 — when nums[left]==nums[mid], we can't determine which half is sorted.

Worst case degrades to O(n) (just left++ to skip the duplicate).

MEDIUM | Binary Search | LeetCode #34 | 2 of 21 | 2/21

Find First and Last Position of Element in Sorted Array

QUESTION

Given an array of integers nums sorted in non-decreasing order, find the starting and ending position of a given target value.

If target is not found in the array, return [-1, -1].

You must write an algorithm with O(log n) runtime complexity.

Example 1	
Input	`nums = [5,7,7,8,8,10], target = 8`
Output	`[3,4]`

Example 2	
Input	`nums = [5,7,7,8,8,10], target = 6`
Output	`[-1,-1]`

Example 3	
Input	`nums = [], target = 0`
Output	`[-1,-1]`

CONSTRAINTS

- $0 <= nums.length <= 10^5$
- $10^9 <= nums[i] <= 10^9$
- nums is a non-decreasing array.
- $10^9 <= target <= 10^9$

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Sorted array with duplicates. Find the first and last index of target. Return [-1,-1] if absent.

Must be O(log n). Right?"

"[5,7,7,8,8,10], target=8 -> [3,4] ok. target=6 -> [-1,-1] ok."

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
scan from the left for the first index of target and
    from the right for the last index
```

TIME	SPACE
O(n)	O(1)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

Ignores that the array is sorted. Two binary searches (lower bound and upper bound) are O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two binary searches — O(log n). Walk-through:"

`leftmost(target): find first index where nums[mid] >= target.`

lo=0, hi=n. While lo<hi: mid=(lo+hi)//2. If nums[mid]<target: lo=mid+1. Else: hi=mid.

If lo==n or nums[lo]!=target: return -1. Else return lo.

`rightmost(target): find first index where nums[mid] > target, then subtract 1.`

lo=0, hi=n. While lo<hi: mid=(lo+hi)//2. If nums[mid]<=target: lo=mid+1. Else: hi=mid.

`return lo-1.`

Return [leftmost, rightmost].

WHY upper bound - 1: bisect_right gives the insert position after last occurrence;

subtracting 1 gives the last occurrence.

WHY hi=n (not n-1): allows the pointer to land on the 'not found' sentinel position.

Time: O(log n). Space: O(1).

PHASE 4 CLEAN CODE

```python
class _34_FirstLastPosition:
    def searchRange(self, nums: List[int], target: int) ->
 List[int]:
        def lower_bound() -> int:
            left, right = 0, len(nums)
            while left < right:
                mid = (left + right) // 2
                if nums[mid] >= target:
                    right = mid
                else:
                    left = mid + 1
            return left

        def upper_bound() -> int:
            left, right = 0, len(nums)
            while left < right:
                mid = (left + right) // 2
                if nums[mid] > target:
                    right = mid
                else:
                    left = mid + 1
            return left

        first = lower_bound()
        if first == len(nums) or nums[first] != target:
            return [-1, -1]
        return [first, upper_bound() - 1]
```

PHASE 5 TEST & DEBUG

"Let me trace searchRange([5,7,7,8,8,10], target=8)."

Find leftmost 8: lo=0,hi=5. mid=2: nums[2]=7<8 -> lo=3.

lo=3,hi=5. mid=4: nums[4]=8==8 AND target==8 -> hi=3 (keep searching left).

lo=3,hi=3. mid=3: nums[3]=8==8 -> hi=2. lo=3>hi=2 -> left=3.

Find rightmost 8: lo=0,hi=5. mid=2: 7<8->lo=3. lo=3,hi=5: mid=4: 8==8->lo=5. lo=5,hi=5: mid=5: 10>8->hi=4. lo=5>hi=4->right=4.

Return [3,4] ok

WHY two separate binary searches: finding leftmost requires hi=mid when equal; rightmost requires lo=mid+1. Same pass can't do both.

Return [-1,-1] if target not found at all: lo=0, run left-binary-search -> check if nums[left]==target first.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n)	**O(1)**

lower_bound and upper_bound are the two fundamental binary search variants — know both.

They correspond to bisect_left and bisect_right in Python's bisect module.

The check 'nums[first] != target' after lower_bound handles the not-found case cleanly.

Given more time I'd ask: what if we want the count of target occurrences?

`upper_bound() - lower_bound() in O(log n).`

MEDIUM | Binary Search | LeetCode #81 | 3 of 21 | 3/21

Search in Rotated Sorted Array II

QUESTION

There is an integer array nums sorted in non-decreasing order (not necessarily with distinct values).

Before being passed to your function, nums is rotated at an unknown pivot index k (0 <= k < nums.length) such that the resulting array is [nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]] (0-indexed). For example, [0,1,2,4,4,4,5,6,6,7] might be rotated at pivot index 5 and become [4,5,6,6,7,0,1,2,4,4].

Given the array nums after the rotation and an integer target, return true if target is in nums, or false if it is not in nums.

You must decrease the overall operation steps as much as possible.

Example 1

Input	`nums = [2,5,6,0,0,1,2], target = 0`
Output	`true`

Example 2

Input	`nums = [2,5,6,0,0,1,2], target = 3`
Output	`false`

CONSTRAINTS

- 1 <= nums.length <= 5000
- 10^4 <= nums[i] <= 10^4
- nums is guaranteed to be rotated at some pivot.
- 10^4 <= target <= 10^4
- Follow up: This problem is similar to Search in Rotated Sorted Array, but nums may contain duplicates. Would this affect the runtime complexity? How and why?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Like Search in Rotated Sorted Array I but NOW the array may contain duplicates.

Return true if target exists. Right?"

"A few quick questions:

Duplicates make it impossible to always determine which half is sorted? Yes.

Worst case degrades to O(n) — unavoidable when nums[left]==nums[mid]==nums[right]."

"Let me walk: nums=[2,5,6,0,0,1,2], target=0 -> true ok. target=3 -> false ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    if nums[i] == target:
        return True
return False
```

TIME	SPACE
O(n)	O(1)

NOTE

Not much slower in the worst case: duplicates can hide which half is sorted, so even the modified binary search degrades to O(n) there. With few duplicates it is O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Binary search with rotated + duplicate handling — O(log n) avg, O(n) worst. Walk-through:"

lo=0, hi=n-1. While lo<=hi:

mid=(lo+hi)//2. If nums[mid]==target: return True.

If nums[lo]==nums[mid]==nums[hi]: lo++; hi--; continue. <- can't determine order

If nums[lo] <= nums[mid]: <- left half sorted

If nums[lo] <= target < nums[mid]: hi=mid-1. Else: lo=mid+1.

Else: <- right half sorted

If nums[mid] < target <= nums[hi]: lo=mid+1. Else: hi=mid-1.

Return False.

WHY lo==mid==hi edge case: duplicates prevent us from knowing which side is sorted.

Shrink both sides by 1 (O(1) per step, but can degrade to O(n) if all duplicates).

WHY still works with rotation: one half is always fully sorted -> binary search on that half.

Time: O(log n) average, O(n) worst. Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _81_SearchRotatedSortedArrayII:
    def search(self, nums, target):
        left, right = 0, len(nums) - 1
        while left <= right:
            mid = (left + right) // 2
            if nums[mid] == target:
                return True
            if nums[left] == nums[mid] == nums[right]:
                left += 1; right -= 1
            elif nums[left] <= nums[mid]:
                if nums[left] <= target < nums[mid]: right
 = mid - 1
```

```
                else: left = mid + 1
            else:
                if nums[mid] < target <= nums[right]: left
= mid + 1
                else: right = mid - 1
        return False
```

PHASE 5 TEST & DEBUG

"Let me trace the monotonic condition and show how each mid update narrows the search."

nums=[2,5,6,0,0,1,2], target=0:

l=0,r=6,mid=3(0)==target -> True ok

nums=[1,1,3,1], target=3: l=0,r=3,mid=1(1). left=right=1(dup) -> l=1,r=2.

mid=1(1),left<=mid: target=3 not in [1,1) -> right side: l=2. mid=2(3)==target -> True ok

The duplicate shortcut (left++,right--) is safe — we only skip confirmed non-targets

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n) avg, O(n) worst (all same values)	**O(1)**

Follow-up: Find Minimum in Rotated Sorted Array II (#154) — same duplicate-handling trick.

Follow-up: recover the original sorted array — find pivot + concatenate.

MEDIUM | Binary Search | LeetCode #153 | 4 of 21 | 4/21

Find Minimum in Rotated Sorted Array

QUESTION

Suppose an array of length n sorted in ascending order is rotated between 1 and n times. For example, the array nums = [0,1,2,4,5,6,7] might become:

CONSTRAINTS

- [4,5,6,7,0,1,2] if it was rotated 4 times.
- [0,1,2,4,5,6,7] if it was rotated 7 times.

Notice that rotating an array [a[0], a[1], a[2], ..., a[n-1]] 1 time results in the array [a[n-1], a[0], a[1], a[2], ..., a[n-2]].

Given the sorted rotated array nums of unique elements, return the minimum element of this array.

You must write an algorithm that runs in O(log n) time.

Example 1

Input	`nums = [3,4,5,1,2]`
Output	`1`
Explanation	*The original array was [1,2,3,4,5] rotated 3 times.*

Example 2

Input	`nums = [4,5,6,7,0,1,2]`
Output	`0`
Explanation	*The original array was [0,1,2,4,5,6,7] and it was rotated 4 times.*

Example 3

Input	`nums = [11,13,15,17]`
Output	`11`
Explanation	*The original array was [11,13,15,17] and it was rotated 4 times.*

- n == nums.length
- 1 <= n <= 5000
- 5000 <= nums[i] <= 5000
- All the integers of nums are unique.
- nums is sorted and rotated between 1 and n times.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Array sorted then rotated. Find the minimum in O(log n). All values unique. Right?

The minimum is at the rotation point — where the array 'wraps'."

"[3,4,5,1,2] -> 1 ok. [4,5,6,7,0,1,2] -> 0 ok. [11,13,15,17] -> 11 (no rotation) ok."

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
return the minimum of the array by scanning every
    element
```

TIME	SPACE
O(n)	O(1)

WHY IT IS SLOW

Ignores that the array was sorted before rotating. Compare the middle with the right end to see which half holds the drop, and binary search: O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Binary search on the rotation point.

If nums[mid] > nums[right]: minimum is in the RIGHT half (mid+1 to right).

Else: minimum is in the LEFT half including mid (left to mid).

When left==right, we've found the minimum.

Time: O(log n). Space: O(1)."

Key insight: one half of a rotated sorted array is always normally sorted — comparing nums[mid] with nums[right] identifies which half contains the minimum in O(1), halving the search each step.

PHASE 4 CLEAN CODE

```python
class _153_FindMinRotated:
    def findMin(self, nums: List[int]) -> int:
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) // 2
            if nums[mid] > nums[right]:
                left = mid + 1          # minimum is to th
e right of mid
            else:
                right = mid             # minimum is mid o
r to its left
        return nums[left]
```

PHASE 5 TEST & DEBUG

"Let me trace findMin([3,4,5,1,2])."

lo=0, hi=4. mid=2. nums[2]=5. 5>nums[4]=2 -> min is in right half -> lo=3.

lo=3, hi=4. mid=3. nums[3]=1 <= nums[4]=2 -> min is in left half including mid -> hi=3.

lo=3, hi=3 -> return nums[3]=1 ok

[4,5,6,7,0,1,2]: lo=0,hi=6. mid=3: 7>nums[6]=2 -> lo=4. lo=4,hi=6: mid=5: 1<=2 -> hi=5. lo=4,hi=5: mid=4: 0<=1 -> hi=4. lo=4,hi=4: return 0 ok

WHY compare mid to rightmost: if nums[mid]>nums[right], pivot is in right half.

PHASE 5 TEST & DEBUG

[1,2]: lo=0,hi=1->mid=0: nums[0]=1<=nums[1]=2 -> min in left -> hi=0. Return nums[0]=1 ok (not rotated).

Key invariant: if nums[mid]<=nums[hi], the right half is sorted -> min is in left half (<=mid).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n)	**O(1)**

The comparison is always against nums[right] (not nums[left]).

If nums[mid] > nums[right], the right portion must be 'lower' — minimum is there.

If nums[mid] <= nums[right], the right portion is in order — minimum is at mid or left.

Given more time I'd ask: what if there are duplicates?

When nums[mid]==nums[right] we can't decide — fall back to right-- (O(n) worst case).

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Binary Search | LeetCode #162 | 5 of 21 | 5/21

Find Peak Element

QUESTION

A peak element is an element that is strictly greater than its neighbors.

Given a 0-indexed integer array nums, find a peak element, and return its index. If the array contains multiple peaks, return the index to any of the peaks.

You may imagine that nums[-1] = nums[n] = -inf. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in O(log n) time.

Example 1

Input	`nums = [1,2,3,1]`
Output	**2**
Explanation	*3 is a peak element and your function should return the index number 2.*

Example 2

Input	`nums = [1,2,1,3,5,6,4]`
Output	**5**
Explanation	*Your function can return either index number 1 where the peak element is 2, or index number 5 where the peak element is 6.*

CONSTRAINTS

- 1 <= nums.length <= 1000
- 2^{31} <= nums[i] <= 2^{31} - 1
- nums[i] != nums[i + 1] for all valid i.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find any peak element — an element strictly greater than its neighbours.

Array has nums[-1]=nums[n]=-inf conceptually. Return any peak index. Right?"

"A few quick questions:

Multiple peaks exist — return any? Yes. Must be O(log n)? Yes."

"Let me walk: nums=[1,2,3,1] -> peak at index 2 (3>2 and 3>1) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    if nums[i] is greater than both neighbours:
        return i
```

TIME	SPACE
O(n)	O(1)

WHY IT IS SLOW

Linear scan. Binary search that moves toward the larger neighbour always keeps a peak in range: O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) linear scan.

Binary search: compare nums[mid] with nums[mid+1].

If nums[mid] < nums[mid+1]: there's a peak on the right -> left = mid+1.

Else: there's a peak at mid or to the left -> right = mid.

When left==right, that's the peak.

Time: O(log n). Space: O(1)."

Key insight: if nums[mid] < nums[mid+1], the right portion must contain a peak (it rises from mid) — this local comparison makes binary search valid even without global sortedness.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _162_FindPeakElement:
    def findPeakElement(self, nums):
        left, right = 0, len(nums) - 1
        while left < right:
            mid = (left + right) // 2
            if nums[mid] < nums[mid + 1]:
                left = mid + 1
            else:
                right = mid
        return left
```

PHASE 5 TEST & DEBUG

"Let me trace findPeakElement on [1,2,3,1]."

Binary search: lo=0, hi=3

mid=1: nums[1]=2 < nums[2]=3 -> peak is in right half: lo=2

mid=2: nums[2]=3 > nums[3]=1 -> peak is here or left: hi=2

lo==hi=2 -> return 2 ok (nums[2]=3 is a peak)

WHY always converge: if nums[mid] < nums[mid+1], right side has a peak (slope rises).

Edge: [1] -> return 0; [-inf,nums,-inf] boundary means any local max qualifies.

PHASE 5 TEST & DEBUG

Invariant: answer always within [lo, hi].

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n)	O(1)

The key insight: gradient always leads to a peak — following the ascending direction guarantees finding one.

Follow-up: Find Peak Index in Mountain Array (#852) — same binary search pattern.

Follow-up: 2D peak — find a peak in a 2D matrix using row-wise binary search.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Binary Search | LeetCode #300 | 6 of 21 | 6/21

Longest Increasing Subsequence

QUESTION

Given an integer array nums, return the length of the longest strictly increasing subsequence.

Example 1	
Input	`nums = [10,9,2,5,3,7,101,18]`
Output	`4`
Explanation	*The longest increasing subsequence is [2,3,7,101], therefore the length is 4.*

Example 2	
Input	`nums = [0,1,0,3,2,3]`
Output	`4`

Example 3	
Input	`nums = [7,7,7,7,7,7,7]`
Output	`1`

CONSTRAINTS

- 1 <= nums.length <= 2500
- 10^4 <= nums[i] <= 10^4
- Follow up: Can you come up with an algorithm that runs in O(n log(n)) time complexity?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find the length of the longest strictly increasing subsequence (not necessarily contiguous).

Return the length, not the subsequence itself. Right?

Follow-up asks for O(n log n) — so know both approaches."

"[10,9,2,5,3,7,101,18] -> [2,3,7,101] length 4 ok. [7,7,7,7] -> 1 (strictly increasing) ok."

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subsequence in subsequences(nums):
    if it is strictly increasing:
        best = max(best, candidate)
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

2^n subsequences. dp[i] = 1 + the best dp[j] for smaller earlier values is O(n^2); keeping the smallest tail for each length with binary search is O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (O(n log n) patience sorting)

Maintain a 'tails' array where tails[i] is the smallest tail of all LIS of length i+1.

For each number, binary search for the leftmost position in tails where tails[pos] >= num.

If found, replace tails[pos] with num (smaller tail = more room to extend).

If not found, append (the LIS can be extended by 1).

`len(tails) at the end is the LIS length.`

This is patience sorting — tails is always sorted so binary search works."

Time: O(n). Space: O(n).

PHASE 4 CLEAN CODE

```python
import bisect

class _300_LIS:
    def lengthOfLIS(self, nums: List[int]) -> int:
        tails = []
        for num in nums:
            pos = bisect.bisect_left(tails, num)
            if pos == len(tails):
                tails.append(num)
            else:
                tails[pos] = num
        return len(tails)
```

PHASE 5 TEST & DEBUG

"Let me trace LIS on [10,9,2,5,3,7,101,18] using patience sort (tails array)."

tails=[].

10: tails=[10]. 9: replace 10 -> tails=[9]. 2: replace 9 -> tails=[2].

5: append -> tails=[2,5]. 3: replace 5 -> tails=[2,3].

7: append -> tails=[2,3,7]. 101: append -> tails=[2,3,7,101].

18: replace 101 -> tails=[2,3,7,18].

`len(tails)=4 ok`

WHY replace (bisect_left): tails[i] = smallest ending value for LIS of length i+1.

Replacing keeps endings as small as possible -> more room for future extensions.

dp[i]=length of LIS ending at index i. dp[0]=1. dp[i]=max(dp[j]+1 for j<i if nums[j]<nums[i]). tails array implements dp optimized: tails[L-1]=smallest tail of LIS of length L.

PHASE 6 COMPLEXITY & FOLLOW-UP

$O(n^2)$ DP: Time $O(n^2)$, Space $O(n)$. Clean and easy to explain.

O(n log n) patience sort: Time O(n log n), Space O(n).

Note: tails does NOT store the actual LIS — it's a virtual 'pile' structure.

To reconstruct the actual LIS, track parent pointers in the DP version.

Given more time I'd ask: longest non-decreasing subsequence?

Change bisect_left to bisect_right — allows equal elements to extend the sequence.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Binary Search | LeetCode #362 | 7 of 21 | 7/21

Design Hit Counter

QUESTION

Design a hit counter which counts the number of hits received in the past 5 minutes (i.e., the past 300 seconds).

Your system should accept a timestamp parameter (in seconds granularity), and you may assume that calls are being made to the system in chronological order (i.e., timestamp is monotonically increasing). Several hits may arrive roughly at the same time.

Implement the HitCounter class:

CONSTRAINTS

- HitCounter() Initializes the object of the hit counter system.
- void hit(int timestamp) Records a hit that happened at timestamp (in seconds). Several hits may happen at the same timestamp.
- int getHits(int timestamp) Returns the number of hits in the past 5 minutes from timestamp (i.e., the past 300 seconds).
- $1 <= timestamp <= 2 * 10^9$
- All the calls are being made to the system in chronological order (i.e., timestamp is monotonically increasing).
- At most 300 calls will be made to hit and getHits.
- Follow up: What if the number of hits per second could be huge? Does your design scale?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Record hits at timestamps and query: how many hits in the past 300 seconds?

Calls are chronologically ordered (timestamps only increase). Right?

getHits(t) returns hits with timestamp in (t-300, t] inclusive — past 5 minutes."

"hit(1),hit(2),hit(3): getHits(4)=3. hit(300): getHits(300)=4. getHits(301)=3 (hit@1 expired).

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def hit(t):
    append t to a list
def getHits(t):
    count the timestamps greater than t - 300
```

TIME	SPACE
O(n) per getHits	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Memory grows with every hit and each query scans them all. A queue that drops timestamps older than 300 seconds on each call (or 300 buckets in a ring) keeps O(1) amortised work and bounded memory.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Store timestamps in a deque. On getHits, pop from the left any timestamp <= t-300.

Remaining deque size is the count. hit() is O(1). getHits() is amortised O(1).

Follow-up (huge hits/second): store (timestamp, count) pairs instead of individual timestamps.

Multiple hits at the same second are collapsed into one deque entry.

PHASE 4 CLEAN CODE

Follow-up: grouped version for high-throughput

```python
class _362_HitCounter:
    def __init__(self):
        self.hits: deque = deque()          # stores individual hit timestamps

    def hit(self, timestamp: int) -> None:
        self.hits.append(timestamp)

    def getHits(self, timestamp: int) -> int:
        while self.hits and self.hits[0] <= timestamp - 300:
            self.hits.popleft()
        return len(self.hits)

class _362_HitCounter_Grouped:
    def __init__(self):
        self.hits: deque = deque()          # stores (timestamp, count) pairs

    def hit(self, timestamp: int) -> None:
        if self.hits and self.hits[-1][0] == timestamp:
            ts, cnt = self.hits.pop()
            self.hits.append((ts, cnt + 1))
        else:
            self.hits.append((timestamp, 1))

    def getHits(self, timestamp: int) -> int:
        while self.hits and self.hits[0][0] <= timestamp - 300:
            self.hits.popleft()
        return sum(cnt for _, cnt in self.hits)
```

PHASE 5 TEST & DEBUG

"Let me trace HitCounter: hit(1),hit(2),hit(3),getHits(4),hit(300),getHits(300),getHits(301)."

hits queue: [(1),(2),(3)]

getHits(4): remove timestamps < 4-300=-296 -> none. Count=3 ok

hit(300): queue=[(1),(2),(3),(300)]

PHASE 5 TEST & DEBUG

```
getHits(300): remove <300-300=0 -> none. Count=4 ok
getHits(301): remove <301-300=1 -> remove (1). Count=3
  ok
```

WHY deque: O(1) front removal; sliding window of 300 seconds maintained.

Binary search variant: timestamps array, bisect for window start index.

PHASE 6 COMPLEXITY & FOLLOW-UP

hit: O(1). getHits: O(k) amortised where k = expired entries (each entry popped at most once).

Space: O(n) where n = total hits in the past 300 seconds.

Binary search alternative: store sorted timestamps, binary search for t-300 cutoff — O(log n).

The deque approach is simpler and amortised O(1) total.

Grouped version handles millions of hits/second — key interview follow-up.

Given more time I'd ask: what if calls aren't chronological?

We'd need a sorted structure (sorted list or BST) and can't use the deque eviction trick.

MEDIUM | Binary Search | LeetCode #528 | 8 of 21 | 8/21

Random Pick with Weight

QUESTION

You are given a 0-indexed array of positive integers w where w[i] describes the weight of the ith index.

You need to implement the function pickIndex(), which randomly picks an index in the range [0, w.length - 1] (inclusive) and returns it. The probability of picking an index i is w[i] / sum(w).

CONSTRAINTS

- For example, if w = [1, 3], the probability of picking index 0 is 1 / (1 + 3) = 0.25 (i.e., 25%), and the probability of picking index 1 is 3 / (1 + 3) = 0.75 (i.e., 75%).
- 1 <= w.length <= 10^4
- 1 <= w[i] <= 10^5
- pickIndex will be called at most 10^4 times.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Pick a random index where index i is chosen with probability w[i]/sum(w).

pickIndex() is called up to 10^4 times — so init can be O(n) but pick must be fast. Right?

w=[1,3]: P(0)=25%, P(1)=75%. The actual randomness in the example output is fine."

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
prefix = the cumulative weights
def pick():
    r = a random number below the total
    scan prefix for the first entry above r
```

TIME	SPACE
O(n) per pick	O(n)

WHY IT IS SLOW

A linear scan per pick. Binary search the cumulative weights for the first prefix above r: O(log n) per pick.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Prefix sum + binary search for weighted random — O(log n) per pick. Walk-through:"

Build prefix sums: prefix[i] = sum(w[0..i]).

Total = prefix[-1].

pickIndex(): rand = random.randint(1, total).

Binary search for leftmost prefix[i] >= rand.

Return that index.

WHY prefix sums: weight w[i] means index i should be picked with probability w[i]/total.

The prefix sum creates ranges: [1..w[0]], [w[0]+1..w[0]+w[1]], ... — each of size w[i].

A random number in [1,total] falls in range i with exactly probability w[i]/total.

WHY binary search: O(log n) to find the bucket; O(n) to build prefix once.

Time: O(n) build, O(log n) per pick. Space: O(n).

PHASE 4 CLEAN CODE

```python
import random

class _528_RandomPickWeight:
    def __init__(self, w: List[int]):
        self.prefix = []
        total = 0
        for weight in w:
            total += weight
            self.prefix.append(total)
        self.total = total

    def pickIndex(self) -> int:
        r   = random.random() * self.total
        lo, hi = 0, len(self.prefix) - 1
        while lo < hi:
            mid = (lo + hi) // 2
            if self.prefix[mid] <= r:
                lo = mid + 1
            else:
                hi = mid
        return lo
```

PHASE 5 TEST & DEBUG

"Let me trace WeightedRandom(w=[1,3]) pickIndex()."

prefix sums: [0,1,4] (cumulative: [1,4])

pickIndex(): random in [1,4]; say random=2 -> bisect_left([1,4],2)=1 -> index 1 ok

random=1 -> bisect_left([1,4],1)=0 -> index 0 ok

Weights [1,3] mean index 0 chosen 1/4 of time, index 1 chosen 3/4 of time

WHY prefix sum + binary search: maps uniform random to weighted distribution in O(log n).

Edge: single element -> always return 0.

prefix sums form cumulative distribution; bisect_left finds which bucket random falls in.

PHASE 6 COMPLEXITY & FOLLOW-UP

Init O(n). pickIndex O(log n). Space O(n).

The prefix sum maps weights to contiguous ranges on [0, total).

A random point in that range lands in bucket i with probability w[i]/total.

`bisect.bisect_right(self.prefix, r) is a one-liner alternative using Python's bisect module.`

Given more time I'd ask: what if weights change dynamically?

We'd need a Fenwick tree (BIT (Binary Indexed Tree)) — O(log n) updates AND O(log n) queries.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Binary Search | LeetCode #729 | 9 of 21 | 9/21

My Calendar I

QUESTION

You are implementing a program to use as your calendar. We can add a new event if adding the event will not cause a double booking.

A double booking happens when two events have some non-empty intersection (i.e., some moment is common to both events.).

The event can be represented as a pair of integers startTime and endTime that represents a booking on the half-open interval [startTime, endTime), the range of real numbers x such that startTime <= x < endTime.

Implement the MyCalendar class:

CONSTRAINTS

- MyCalendar() Initializes the calendar object.
- boolean book(int startTime, int endTime) Returns true if the event can be added to the calendar successfully without causing a double booking. Otherwise, return false and do not add the event to the calendar.
- 0 <= start < end <= 10^9
- At most 1000 calls will be made to book.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Book a calendar event [start, end). Return false if it overlaps any existing event. Right?"

"A few quick questions:

Half-open interval [start, end): start inclusive, end exclusive? Yes."

"Let me walk: book(10,20)->T, book(15,25)->F (overlaps), book(20,30)->T ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def book(s, e):
    for e2) in e2)s:
        if s < e2 and s2 < e:
            reject
    otherwise add (s, e) to the list
```

TIME	SPACE
O(n) per booking	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Checks every earlier booking. A sorted structure (balanced tree or sorted list) lets you check only the neighbours at the insertion point: O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Sorted list of bookings with binary search overlap check — O(log n) per booking."

`bookings = sorted list of (start, end) intervals.`

For each new booking (start, end):

Use bisect to find insertion position i (where start would go in sorted order).

Check left neighbor: if i > 0 and bookings[i-1][1] > start -> overlap -> return False.

Check right neighbor: if i < len(bookings) and bookings[i][0] < end -> overlap -> return False.

No overlap: insert (start, end) at position i -> return True.

WHY only check immediate neighbors: sorted list means only adjacent intervals can overlap

with a new interval — all others are guaranteed to be too far left or right.

WHY sorted by start: overlaps with (start, end) can only happen with intervals whose

start < end (left neighbor) or whose start < new_end (right neighbor).

Time: O(log n) binary search + O(n) insert. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import bisect
class _729_MyCalendarI:
    def __init__(self):
        self._starts = []
        self._ends = []

    def book(self, start, end):
        idx = bisect.bisect_left(self._starts, end)
        if idx > 0 and self._ends[idx-1] > start:
            return False
        bisect.insort(self._starts, start)
        bisect.insort(self._ends, end)
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace MyCalendar book(10,20), book(15,25), book(20,30)."

Use sorted structure (BST or sorted list) of booked intervals

`book(10,20): empty -> add [10,20]. Return True ok`

`book(15,25): find overlap: [10,20] overlaps [15,25) since 10<25 and 15<20 -> return False ok`

`book(20,30): [10,20] -> 20==20 (start==end, no overlap) -> add [20,30]. Return True ok`

PHASE 5 TEST & DEBUG

Overlap condition: start < end2 AND start2 < end (not adjacent but intersecting)

WHY BST/sortedlist: O(log n) lookup for overlapping interval in sorted structure.

With SortedList: bisect for overlap check O(log n). Naive linear scan O(n) per booking.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n) query + O(n) insort (shift)	**O(n)**

With a balanced BST (SortedList from sortedcontainers): O(log n) all ops.

Follow-up: My Calendar II (#731) — allow double bookings; track triple bookings.

Follow-up: My Calendar III (#732) — return max k-fold overlap at any point.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Binary Search | LeetCode #731 | 10 of 21 | 10/21

My Calendar II

QUESTION

You are implementing a program to use as your calendar. We can add a new event if adding the event will not cause a triple booking.

A triple booking happens when three events have some non-empty intersection (i.e., some moment is common to all the three events.).

The event can be represented as a pair of integers startTime and endTime that represents a booking on the half-open interval [startTime, endTime), the range of real numbers x such that startTime <= x < endTime.

Implement the MyCalendarTwo class:

CONSTRAINTS

- MyCalendarTwo() Initializes the calendar object.
- boolean book(int startTime, int endTime) Returns true if the event can be added to the calendar successfully without causing a triple booking. Otherwise, return false and do not add the event to the calendar.
- 0 <= start < end <= 10^9
- At most 1000 calls will be made to book.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Book events; return false only if the new event would cause a TRIPLE booking.

Double bookings (overlaps) are allowed. Right?"

"A few quick questions:

Keep track of single and double bookings separately? Yes."

"Let me walk: book(10,20)->T, book(50,60)->T, book(10,40)->T, book(5,15)->F (triple at 10-15) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def book(s, e):
    for (a, b) in pairs(existing):
        if the new event also overlaps both:
            reject (triple booking)
    otherwise add it
```

TIME	SPACE
O(n^2) per booking	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Checks all pairs per booking. Keep a list of bookings and a list of double-booked overlaps: reject if the new event overlaps a double-booking, then add its new overlaps: O(n) per booking.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Difference array / sorted boundaries — O(n log n). Walk-through:"

For each booking, track start/end events.

At booking attempt [start, end):

Build or update overlap count at each point.

If any point has overlap count >= 2: this booking creates a triple-booking -> reject False.

Efficient: maintain list of (start, +1) and (end, -1) events. Sort and scan prefix sum.

If any prefix sum >= 3 after tentative add: reject.

calendar1 = intervals with no double-book. calendar2 = intervals with double-booking (pre-existing overlaps).

Alternative: check if [start,end) overlaps any interval in calendar2.

If not: add; also add intersections with calendar1 to calendar2. Return True.

Time: O(n²) naive, O(n log n) with sorted structure. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _731_MyCalendarII:
    def __init__(self):
        self._single = []
        self._double = []

    def book(self, start, end):
        for s, e in self._double:
            if start < e and end > s:
                return False
        for s, e in self._single:
            if start < e and end > s:
                self._double.append((max(start, s), min(en
d, e)))
        self._single.append((start, end))
        return True
```

PHASE 5 TEST & DEBUG

"Let me trace the monotonic condition and show how each mid update narrows the search."

book(10,20)->single=[(10,20)].
book(50,60)->single+=(50,60).
book(10,40)->overlaps(10,20)->double+=[(10,20)].
single+=(10,40).

book(5,15)->double has (10,20). 5<20 and 15>10->True->return False ok

PHASE 5 TEST & DEBUG

Double list tracks all pairwise overlaps; any triple overlap triggers False

Edge: book(10,20) first -> always True (no overlaps yet) ok. Non-overlapping books always succeed. book(10,20) twice -> double overlap = triple -> False ok.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^2)$ worst case	$O(n)$

Follow-up: My Calendar III (#732) — return k for max k-fold overlap; difference array.

Follow-up: if we need all events that overlap at a given point, use interval tree.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Binary Search | LeetCode #852 | 11 of 21 | 11/21

Peak Index in a Mountain Array

QUESTION

You are given an integer mountain array arr of length n where the values increase to a peak element and then decrease.

Return the index of the peak element.

Your task is to solve it in O(log(n)) time complexity.

Example 1	
Input	`arr = [0,1,0]`
Output	`1`

Example 2	
Input	`arr = [0,2,1,0]`
Output	`1`

Example 3	
Input	`arr = [0,10,5,2]`
Output	`1`

CONSTRAINTS

- 3 <= arr.length <= 10^5
- 0 <= arr[i] <= 10^6
- arr is guaranteed to be a mountain array.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Array guaranteed to have exactly one peak (mountain shape: strictly increasing then decreasing).

Return the index of the peak in O(log n). Right?"

"[0,1,0] -> 1. [0,2,1,0] -> 1. [0,10,5,2] -> 1. ok"

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
return the index of the largest value found by
    scanning every element
```

TIME	SPACE
O(n)	O(1)

WHY IT IS SLOW

Linear scan of a mountain array. Binary search: if arr[mid] < arr[mid + 1] the peak is to the right, else at mid or to the left: O(log n).

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Binary search on mountain peak — O(log n). Walk-through:"

lo=0, hi=n-1. While lo < hi:

`mid = (lo+hi)//2.`

If arr[mid] < arr[mid+1]: lo=mid+1. <- ascending part, peak is to the right

Else: hi=mid. <- descending or at peak, peak is here or left

Return lo.

WHY compare mid and mid+1: determines which side of the mountain we're on.

If arr[mid] < arr[mid+1]: we're in the ascending portion -> peak is right of mid.

If arr[mid] > arr[mid+1]: descending -> peak is at mid or left.

WHY hi=mid (not mid-1): mid itself could be the peak.

Time: O(log n). Space: O(1).

PHASE 4 CLEAN CODE

```python
class _852_PeakIndex:
    def peakIndexInMountainArray(self, arr: List[int]) ->
int:
        left, right = 0, len(arr) - 1
        while left < right:
            mid = (left + right) // 2
            if arr[mid] < arr[mid + 1]:
                left = mid + 1      # still ascending — pe
ak is further right
            else:
                right = mid         # descending or at pea
k — search left including mid
        return left
```

PHASE 5 TEST & DEBUG

"Let me trace peakIndexInMountainArray([0,1,0])."

Binary search: lo=0, hi=2

mid=1: arr[1]=1 > arr[2]=0 -> peak at or left of mid: hi=1

lo==hi=1 -> return 1 ok

WHY binary search works: mountain array has one peak; if arr[mid]>arr[mid+1], peak is left; else right.

Trace [0,2,1,0]: lo=0,hi=3; mid=1: arr[1]=2>arr[2]=1 -> hi=1; lo==hi=1 -> return 1 ok

Edge: guaranteed mountain array with valid peak; no need to check boundaries.

Mountain property guarantees convergence; mid always closer to peak -> terminates in O(log n).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(log n)	O(1)

The comparison arr[mid] vs arr[mid+1] acts as a gradient — positive means 'go right', negative means 'go left'.

This is the same logic as Find Peak Element (#162) — the general version where the array isn't guaranteed

to be a full mountain. Mention the connection.

Given more time I'd ask: what if there are multiple peaks?

Same algorithm finds one peak. For all peaks, we'd need a linear scan.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Binary Search | LeetCode #875 | 12 of 21 | 12/21

Koko Eating Bananas

QUESTION

Koko loves to eat bananas. There are n piles of bananas, the ith pile has piles[i] bananas. The guards have gone and will come back in h hours.

Koko can decide her bananas-per-hour eating speed of k. Each hour, she chooses some pile of bananas and eats k bananas from that pile. If the pile has less than k bananas, she eats all of them instead and will not eat any more bananas during this hour.

Koko likes to eat slowly but still wants to finish eating all the bananas before the guards return.

Return the minimum integer k such that she can eat all the bananas within h hours.

Example 1	
Input	`piles = [3,6,7,11], h = 8`
Output	`4`

Example 2	
Input	`piles = [30,11,23,4,20], h = 5`
Output	`30`

Example 3	
Input	`piles = [30,11,23,4,20], h = 6`
Output	`23`

CONSTRAINTS

- 1 <= piles.length <= 10^4
- piles.length <= h <= 10^9
- 1 <= piles[i] <= 10^9

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Koko eats at a constant speed k bananas/hour from piles. If a pile has fewer than k, she finishes it

and waits the rest of the hour. Find the minimum k to eat all piles within h hours. Right?"

"A few quick questions:

k must be a positive integer? Yes. h >= len(piles) guaranteed (always feasible)."

"Let me walk: piles=[3,6,7,11], h=8 -> k=4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for speed k = 1, 2, 3 ...:
    if every pile can be eaten within h hours at
        speed k:
        return k
```

TIME	SPACE
O(max(piles) * n)	O(1)

WHY IT IS SLOW

Tries every speed up to the largest pile. Hours needed fall as speed rises, so binary search on k with an O(n) check is O(n log max).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Binary search on eating speed k — O(n log W). Walk-through:"

Binary search lo=1, hi=max(piles). Answer = smallest k where Koko can finish in h hours.

`can_finish(k): hours = sum(ceil(p/k) for p in piles). Return hours <= h.`

`ceil(p/k) = (p+k-1)//k in integer math.`

If can_finish(mid): hi=mid (try smaller speed). Else: lo=mid+1 (need faster).

WHY monotone: if speed k works, speed k+1 also works -> binary search valid.

WHY lo converges to answer: lo is always a valid lower bound; when lo==hi both point

to the minimum valid speed.

Time: O(n log W) where W=max pile. Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
import math
class _875_KokoEatingBananas:
    def minEatingSpeed(self, piles, h):
        def hours_needed(k):
            return sum(math.ceil(p / k) for p in piles)
        lo, hi = 1, max(piles)
        while lo < hi:
            mid = (lo + hi) // 2
            if hours_needed(mid) <= h:
                hi = mid
            else:
                lo = mid + 1
        return lo
```

PHASE 5 TEST & DEBUG

"Let me trace minEatingSpeed([3,6,7,11], h=8)."

Binary search on k (eating speed): lo=1, hi=11

mid=6: hours=ceil(3/6)+ceil(6/6)+ceil(7/6)+ceil(11/6)=1+1+2+2=6<=8 -> hi=6

mid=3: hours=ceil(3/3)+ceil(6/3)+ceil(7/3)+ceil(11/3)=1+2+3+4=10>8 -> lo=4

mid=5: hours=1+2+2+3=8<=8 -> hi=5

mid=4: hours=1+2+2+3=8 wait: ceil(3/4)=1,ceil(6/4)=2,ceil(7/4)=2,ceil(11/4)=3=8<=8 -> hi=4

lo==hi=4 -> return 4 ok

WHY binary search: monotone — faster eating -> fewer hours needed.

PHASE 6 COMPLEXITY & FOLLOW-UP

Time O(n log(max(piles))). Space O(1).

Follow-up: Capacity to Ship Packages Within D Days (#1011) — identical binary-search-on-answer pattern.

Follow-up: Split Array Largest Sum (#410) — same technique, different feasibility check.

MEDIUM | Binary Search | LeetCode #981 | 13 of 21 | 13/21

Time Based Key-Value Store

QUESTION

Design a time-based key-value data structure that can store multiple values for the same key at different time stamps and retrieve the key's value at a certain timestamp.

Implement the TimeMap class:

CONSTRAINTS

- TimeMap() Initializes the object of the data structure.
- void set(String key, String value, int timestamp) Stores the key key with the value value at the given time timestamp.
- String get(String key, int timestamp) Returns a value such that set was called previously, with timestamp_prev <= timestamp. If there are multiple such values, it returns the value associated with the largest timestamp_prev. If there are no values, it returns "".
- 1 <= key.length, value.length <= 100
- key and value consist of lowercase English letters and digits.
- 1 <= timestamp <= 10^7
- All the timestamps timestamp of set are strictly increasing.
- At most 2 * 10^5 calls will be made to set and get.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Key-value store where each key can have multiple values at different timestamps.

set(key, value, ts) stores the entry. get(key, ts) returns the value with the

largest timestamp <= ts. Return '' if none exists. Right?

set timestamps are strictly increasing — so we can binary search the history list."

"set('foo','bar',1). get('foo',3) -> 'bar' (no ts<=3 except 1). ok"

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def set(key, value, t):
    append (t, value) to the key's list
def get(key, t):
    scan the list for the entry with the largest
        timestamp <= t
```

TIME	SPACE
O(n) per get	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Scans all of a key's values. Timestamps arrive in increasing order, so the list is sorted and a binary search gives O(log n) per get.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"HashMap of sorted timestamp lists + binary search — O(log n) per get. Walk-through:"

set(key, value, timestamp):
store[key].append((timestamp, value)). Always increasing t -> sorted.

get(key, timestamp):

If key not in store: return ''.

List of (ts, val) sorted by ts (always, since set called with increasing timestamps).

bisect_right on timestamps: find rightmost ts <= timestamp.

Use bisect_right on the ts values: idx = bisect_right(store[key], (timestamp, chr(255))) - 1.

If idx >= 0: return store[key][idx][1]. Else: return ''.

WHY bisect_right then -1: get the last entry with ts <= target timestamp.

Time: O(1) set, O(log n) get. Space: O(n).

PHASE 4 CLEAN CODE

```
class _981_TimeMap:
    def __init__(self):
        self.store: defaultdict = defaultdict(list)     # key -> [(ts, value), ...]

    def set(self, key: str, value: str, timestamp: int) -> None:
        self.store[key].append((timestamp, value))

    def get(self, key: str, timestamp: int) -> str:
        entries = self.store[key]
        lo, hi  = 0, len(entries) - 1
        result  = ''
        while lo <= hi:
            mid = (lo + hi) // 2
            if entries[mid][0] <= timestamp:
                result = entries[mid][1]    # valid candidate — try to go right for a larger ts
                lo = mid + 1
            else:
                hi = mid - 1
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the monotonic condition and show how each mid update narrows the search."

set('foo','bar',1): store['foo']=[(1,'bar')].

get('foo',1): entries=[(1,'bar')]. mid=0, ts=1<=1 -> result='bar', lo=1. lo>hi -> return 'bar' ok

get('foo',3): mid=0, ts=1<=3 -> result='bar'. lo=1>hi=0 -> return 'bar' ok

set('foo','bar2',4): store['foo']=[(1,'bar'),(4,'bar2')].

PHASE 5 TEST & DEBUG

```
get('foo',4): mid=0(1<=4)->result='bar',lo=1.
  mid=1(4<=4)->result='bar2',lo=2. return 'bar2' ok
get('foo',5): same -> 'bar2' ok
```

PHASE 6 COMPLEXITY & FOLLOW-UP

set O(1). get O(log n). Space O(total entries).

The 'find rightmost ts <= query' is a standard upper_bound - 1 binary search pattern.

Timestamps strictly increasing -> no sorting needed — the list maintains order automatically.

bisect.bisect_right alternative: pos = bisect.bisect_right(entries, (timestamp, chr(127))) - 1

(appending chr(127) to go past any value at that exact timestamp).

Given more time I'd ask: what if set timestamps aren't guaranteed to be increasing?

We'd need to sort the list on insert or use a sorted data structure.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Binary Search | LeetCode #1011 | 14 of 21 | 14/21

Capacity to Ship Packages Within D Days

QUESTION

A conveyor belt has packages that must be shipped from one port to another within days days.

The ith package on the conveyor belt has a weight of weights[i]. Each day, we load the ship with packages on the conveyor belt (in the order given by weights). We may not load more weight than the maximum weight capacity of the ship.

Return the least weight capacity of the ship that will result in all the packages on the conveyor belt being shipped within days days.

Example 1

Input	`weights = [1,2,3,4,5,6,7,8,9,10], days = 5`
Output	**15**
Explanation	*A ship capacity of 15 is the minimum to ship all the packages in 5 days like this:* *1st day: 1, 2, 3, 4, 5* *2nd day: 6, 7* *3rd day: 8* *4th day: 9* *5th day: 10* *Note that the cargo must be shipped in the order given, so using a ship of capacity 14 and splitting the packages into parts like (2, 3, 4, 5), (1, 6, 7), (8), (9), (10) is not allowed.*

Example 2

Input	`weights = [3,2,2,4,1,4], days = 3`
Output	**6**
Explanation	*A ship capacity of 6 is the minimum to ship all the packages in 3 days like this:* *1st day: 3, 2* *2nd day: 2, 4* *3rd day: 1, 4*

Example 3

Input	`weights = [1,2,3,1,1], days = 4`
Output	**3**
Explanation	*1st day: 1* *2nd day: 2* *3rd day: 3* *4th day: 1, 1*

CONSTRAINTS

- 1 <= days <= weights.length <= $5 * 10^4$
- 1 <= weights[i] <= 500

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find the minimum ship capacity that lets us ship all packages in at most D days.

Packages must be shipped in order — we can't reorder them. Right?

Capacity must be at least max(weights) (to handle any single package).

Capacity at most sum(weights) (ship everything in one day)."

"weights=[1..10], days=5 -> 15 ok (greedy daily assignment: 1-5, 6-7, 8, 9, 10)"

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for capacity in
    range(max(weights)_up,_sum(weights)_+_1):
    simulate loading in order
return the first capacity that ships within D days
```

TIME	SPACE
O(n * (sum - max))	O(1)

WHY IT IS SLOW

Tries every capacity. Feasibility is monotonic in capacity, so binary search on it with an O(n) simulation is O(n log(sum)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Binary search on the answer (capacity).

The feasibility function 'can_ship(capacity)' is monotone: if C works, C+1 also works.

So binary search for the leftmost capacity where can_ship returns True.

can_ship: greedily fill each day until the load would exceed capacity, then start a new day.

Count days needed. Return days <= D.

Time: O(n log(sum(weights))). Space: O(1).

PHASE 4 CLEAN CODE

```python
class _1011_CapacityShip:
    def shipWithinDays(self, weights: List[int], days: int
) -> int:
        def can_ship(capacity: int) -> bool:
            day_load = 0
            days_used = 1
            for w in weights:
                if day_load + w > capacity:
                    days_used += 1
                    day_load = 0
                day_load += w
```

```
            return days_used <= days

        lo, hi = max(weights), sum(weights)
        while lo < hi:
            mid = (lo + hi) // 2
            if can_ship(mid):
                hi = mid        # valid — try smaller
            else:
                lo = mid + 1    # not valid — need more ca
pacity
        return lo
```

PHASE 5 TEST & DEBUG

"Let me trace shipWithinDays([1,2,3,4,5,6,7,8,9,10], days=5)."

lo=10 (max single weight), hi=55 (total weight). Binary search on capacity.

cap=32: simulate: day totals [1+2+...]=10,11,12? sum greedily.

cap=15: days=[1+2+3+4+5=15],[6+7=13]... wait items=[1..10].

[1+2+3+4+5]=15ok,[6+7]=13ok,[8]=8ok,[9]=9ok,[10]=10ok -> 5 days ok -> try lower.

cap=10: [1+2+3+4]=10,[5]=5+need more? [5+6=11>10]->[5],[6],[7],[8],[9],[10]->7 days>5 .

Binary search converges to cap=15 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log(sum))	**O(1)**

This is 'binary search on the answer' — a powerful pattern where the answer space is

a range of integers and feasibility is monotone.

The feasibility check is a greedy simulation — always load greedily to avoid underusing capacity.

Same pattern appears in: Koko Eating Bananas (#875), Split Array Largest Sum (#410).

Given more time I'd ask: what if packages can be reordered?

Then we'd try to optimise placement — different problem, potentially NP (Non-deterministic Polynomial)-hard.

MEDIUM | Binary Search | LeetCode #1027 | 15 of 21 | 15/21

Longest Arithmetic Subsequence

QUESTION

Given an array nums of integers, return the length of the longest arithmetic subsequence in nums.

Note that:

CONSTRAINTS

- A subsequence is an array that can be derived from another array by deleting some or no elements without changing the order of the remaining elements.
- A sequence seq is arithmetic if seq[i + 1] - seq[i] are all the same value (for 0 <= i < seq.length - 1).

Example 1

Input	`nums = [3,6,9,12]`
Output	`4`
Explanation	*The whole array is an arithmetic sequence with steps of length = 3.*

Example 2

Input	`nums = [9,4,7,2,10]`
Output	`3`
Explanation	*The longest arithmetic subsequence is [4,7,10].*

Example 3

Input	`nums = [20,1,15,3,10,5,8]`
Output	`4`
Explanation	*The longest arithmetic subsequence is [20,15,10,5].*

- 2 <= nums.length <= 1000
- 0 <= nums[i] <= 500

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the longest arithmetic subsequence: elements equally spaced (constant difference). Right?"

"A few quick questions:

Return the length, not the subsequence itself? Yes. At least length 2? Yes."

"Let me walk: nums=[3,6,9,12] -> diff=3, length=4 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for subsequence in subsequences(nums):
    if its differences are all equal:
        best = max(best, candidate)
```

TIME	SPACE
O(2^n * n)	O(n)

WHY IT IS SLOW

2^n subsequences. dp[i][d] = 1 + dp[j][d] for earlier j with nums[i] - nums[j] = d, kept in a hash map per index, is O(n^2).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"DP with per-node difference tracking — O(n²). Walk-through:"

`dp[i][diff] = length of longest arithmetic subsequence ending at index i with common diff 'diff'.`

For each j < i:

`d = nums[i] - nums[j].`

`dp[i][d] = dp[j].get(d, 1) + 1`

<- extend subsequence ending at j with difference d, or start a new one of length 2.

`best = max(best, dp[i][d]).`

WHY hash map per index: differences can be negative or large — dict is more flexible than array.

WHY initialize to 1: a single element is always an AP (Arithmetic Progression) of length 1; two elements is length 2.

Time: O(n²). Space: O(n²) worst case.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1027_LongestArithmeticSubsequence:
    def longestArithSeqLength(self, nums):
        n = len(nums)
        dp = [dict() for _ in range(n)]
        best = 2
        for i in range(1, n):
            for j in range(i):
                diff = nums[i] - nums[j]
                dp[i][diff] = dp[j].get(diff, 1) + 1
                best = max(best, dp[i][diff])
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace the monotonic condition and show how each mid update narrows the search."

nums=[3,6,9,12]: diff=3 at each step. dp[1]={3:2}, dp[2]={3:3}, dp[3]={3:4}. best=4 ok

PHASE 5 TEST & DEBUG

nums=[9,4,7,2,10]: various diffs. best=3([4,7,10] or [9,4,-1]?) -> 3 ok

Dp[i][diff] = length of AP ending at nums[i] with common difference diff

Edge: nums=[0,0] -> diff=0, dp[1]={0:2}, answer=2 ok. Single element -> 1. Negative diffs handled: diff can be any integer (stored in dict).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^2)$	$O(n^2)$ for dicts

Follow-up: Arithmetic Slices (#413) — contiguous arithmetic subsequences.

Follow-up: Longest Arithmetic Subsequence of Given Difference (#1218) — fixed diff, O(n).

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Binary Search | LeetCode #1060 | 16 of 21 | 16/21

Missing Element in Sorted Array

QUESTION

Given an integer array nums which is sorted in ascending order and all of its elements are unique and given also an integer k, return the kth missing number starting from the leftmost number of the array.

Example 1	
Input	`nums = [4,7,9,10], k = 1`
Output	**5**
Explanation	*The first missing number is 5.*

Example 2	
Input	`nums = [4,7,9,10], k = 3`
Output	**8**
Explanation	*The missing numbers are [5,6,8,...], hence the third missing number is 8.*

Example 3	
Input	`nums = [1,2,4], k = 3`
Output	**6**
Explanation	*The missing numbers are [3,5,6,7,...], hence the third missing number is 6.*

CONSTRAINTS

- 1 <= nums.length <= $5 * 10^4$
- 1 <= nums[i] <= 10^7
- nums is sorted in ascending order, and all the elements are unique.
- 1 <= k <= 10^8
- Follow up: Can you find a logarithmic time complexity (i.e., O(log(n))) solution?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Sorted unique integers. Find the k-th missing integer starting from nums[0].

Missing count at index i = (nums[i] - nums[0]) - i (how many numbers should exist but don't).

Follow-up asks for O(log n). Right?"

"[4,7,9,10], k=1: missing = [5,6,8,...]. 1st missing = 5 ok

[4,7,9,10], k=3: [5,6,8,...]. 3rd = 8 ok"

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
walk through the array counting how many numbers are
    missing between neighbours until the k-th missing
    one is passed
```

TIME	SPACE
O(n)	O(1)

WHY IT IS SLOW

Linear scan. missing(i) = nums[i] - nums[0] - i is non-decreasing, so binary search the first index whose missing count reaches k: O(log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (O(log n))

"Binary search on missing count — O(log n). Walk-through:"

`missing(mid) = nums[mid] - nums[0] - mid (how many numbers are missing in nums[0..mid]).`

If nums were complete (no gaps): nums[mid] = nums[0] + mid -> missing = 0.

Each gap of size g contributes g-1 missing numbers.

Binary search: find the rightmost mid where missing(mid) < k.

Answer = nums[mid] + (k - missing(mid)). (jump from nums[mid] to the k-th missing)

WHY missing(mid) = nums[mid]-nums[0]-mid: elements nums[0],nums[0]+1,...,nums[0]+mid would be

present with no gaps. Extra elements in range not in array = missing count.

Time: O(log n). Space: O(1).

PHASE 4 CLEAN CODE

```python
class _1060_MissingElement:
    def missingElement(self, nums: List[int], k: int) -> i
nt:
        def missing(i: int) -> int:
            return (nums[i] - nums[0]) - i

        n = len(nums)
        if k > missing(n - 1):
            return nums[-1] + (k - missing(n - 1))

        lo, hi = 0, n - 1
        while lo < hi:
            mid = (lo + hi) // 2
            if missing(mid) < k:
                lo = mid + 1        # not enough missing y
et — go right
            else:
                hi = mid            # too many or just eno
ugh — go left
        return nums[lo - 1] + (k - missing(lo - 1))
```

PHASE 5 TEST & DEBUG

"Let me trace the monotonic condition and show how each mid update narrows the search."

nums=[4,7,9,10], k=1:

missing: [0]=0, [1]=2, [2]=3, [3]=4.

k=1<=missing(3)=4. Binary search: lo=0,hi=3,mid=1. missing(1)=2>=1->hi=1.

lo=0,hi=1,mid=0. missing(0)=0<1->lo=1. lo=hi=1.

`return nums[0]+(1-missing(0))=4+(1-0)=5 ok`

nums=[4,7,9,10], k=3:

`missing(3)=4>=3. Binary search -> lo lands at 2. missing(1)=2<3->lo=2. lo=hi=2.`

`return nums[1]+(3-missing(1))=7+(3-2)=8 ok`

k=5: missing(3)=4<5 -> return nums[3]+(5-4)=10+1=11 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

O(log n) with binary search. O(1) space.

The missing(i) formula is the key: (nums[i]-nums[0])-i counts how many integers

in [nums[0], nums[i]] are absent from the array.

The past-the-end check handles k > total missing numbers in the array gracefully.

Given more time I'd ask: what if there are duplicate values in nums?

`missing(i) would overcount — we'd need to adjust for duplicates.`

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Binary Search | LeetCode #1062 | 17 of 21 | 17/21

Longest Repeating Substring

QUESTION

Given a string s, return the length of the longest repeating substrings. If no repeating substring exists, return 0.

Example 1

Input	`s = "abcd"`
Output	`0`
Explanation	*There is no repeating substring.*

Example 2

Input	`s = "abbaba"`
Output	`2`
Explanation	*The longest repeating substrings are "ab" and "ba", each of which occurs twice.*

Example 3

Input	`s = "aabcaabdaab"`
Output	`3`
Explanation	*The longest repeating substring is "aab", which occurs 3 times.*

CONSTRAINTS

- 1 <= s.length <= 2000
- s consists of lowercase English letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Find the length of the longest substring that appears at least twice in s.

The two occurrences can overlap. Return 0 if no repeating substring. Right?"

"'abcd'->0. 'abbaba'->2 ('ab','ba'). 'aabcaabdaab'->3 ('aab')."

Can the tree be empty (root=null)? Are all node values unique? Right?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for length L from n - 1 down to 1:
    put every substring of that length in a set
    if one repeats:
        return L
```

TIME	SPACE
O(n^3)	O(n^2)

PHASE 2 BRUTE FORCE

WHY IT IS SLOW

Hashes every substring of every length. Repeating substrings are monotonic in length (a repeat of length L has repeats of every shorter length), so binary search on L with a rolling hash is O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE (DP approach — simpler to explain)

"Binary search on length + rolling hash — O(n log n). Walk-through:"

Binary search on length L:

`hasRepeat(L): slide window of size L across s, check for duplicate substrings using hash set.`

If True: try longer. Else: try shorter.

Rolling hash: O(n) to check all length-L substrings.

Alternatively: suffix array + LCP array:

Build suffix array in O(n log n).

LCP array: LCP[i] = longest common prefix of suffix_array[i] and suffix_array[i-1].

Answer = max(LCP).

WHY suffix array: directly computes all repeated substrings via LCP values.

Time: O(n log n). Space: O(n).

PHASE 4 CLEAN CODE

```
class _1062_LongestRepeatingSubstring:
    def longestRepeatingSubstring(self, s: str) -> int:
        n    = len(s)
        dp   = [[0] * (n + 1) for _ in range(n + 1)]
        best = 0
        for i in range(1, n + 1):
            for j in range(i + 1, n + 1):      # j > i ens
ures different starting positions
                if s[i - 1] == s[j - 1]:
                    dp[i][j] = dp[i - 1][j - 1] + 1
                    best = max(best, dp[i][j])
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace longestRepeatingSubstring('abcd')."

Binary search on length L. Check if any length-L substring appears twice.

L=2: 'ab','bc','cd' -> all unique -> no repeat

L=1: 'a','b','c','d' -> all unique -> no repeat. Return 0 ok

Trace 'aab': L=1: 'a' appears twice -> has repeat. L=2: 'aa','ab' -> unique -> no repeat. Return 1 ok ('a')

WHY binary search: if L has repeat, L-1 also has repeat (prefix of repeating substring). Monotone.

Rolling hash or suffix array for O(n log n) check.

Binary search + suffix array runs in O(n log² n); useful when n is large.

PHASE 5 TEST & DEBUG

dp[i][j]=LCP(suffix i, suffix j). Or: dp[L]=True if any length-L substring repeats. Binary search L->check. dp[i][j] via suffix array: O(n log n). dp[j]=longest repeat ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
$O(n^2)$	$O(n^2)$ Acceptable for n<=2000 (4M cells)

Binary search + hashing alternative: O(n log n) time, O(n) space — faster but more complex.

Given more time I'd ask: return the actual substring, not just the length?

Track (i,j) when best is updated, then s[i-best:i].

MEDIUM | Binary Search | LeetCode #1146 | 18 of 21 | 18/21

Snapshot Array

QUESTION

Implement a SnapshotArray that supports the following interface:

CONSTRAINTS

- SnapshotArray(int length) initializes an array-like data structure with the given length. Initially, each element equals 0.
- void set(index, val) sets the element at the given index to be equal to val.
- int snap() takes a snapshot of the array and returns the snap_id: the total number of times we called snap() minus 1.
- int get(index, snap_id) returns the value at the given index, at the time we took the snapshot with the given snap_id

Example 1

Input	`["SnapshotArray","set","snap","set","get"]` `[[3],[0,5],[],[0,6],[0,0]]`
Output	`[null,null,0,null,5]`

Explanation	*SnapshotArray snapshotArr = new SnapshotArray(3); // set the length to be 3*
	snapshotArr.set(0,5); // Set array[0] = 5
	snapshotArr.snap(); // Take a snapshot, return snap_id = 0
	snapshotArr.set(0,6);
	snapshotArr.get(0,0); // Get the value of array[0] with snap_id = 0, return 5

- $1 <= length <= 5 * 10^4$
- $0 <= index < length$
- $0 <= val <= 10^9$
- 0 <= snap_id < (the total number of times we call snap())
- At most $5 * 10^4$ calls will be made to set, snap, and get.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

`set(index, val) updates the array. snapshot() takes a snapshot. get(index, snap_id) returns the value at index at snap_id. Right?"`

"A few quick questions:

`get() returns value from a past snapshot, not current state? Yes."`

"Let me walk: set(0,5), snap()->0, set(0,6), get(0,0)->5 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

PHASE 2 BRUTE FORCE

```
def snap():
    copy the whole array into a list of snapshots
    set and get(index, snap_id) read or write the
        arrays
```

TIME	SPACE
O(n) per snap	O(n * snaps)

WHY IT IS SLOW

Every snapshot copies all n values. For each index keep only a list of (snap id, value) changes; snap just increments the id, and get binary searches the index's list: snap O(1), get O(log s).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"HashMap of index->sorted list of (timestamp, value), binary search for snap — O(log s). Walk-through:"

`snap_id = 0. store[index] = sorted list of (snap_id, value).`

`set(index, val): update or append current snap_id and val for this index.`

`snap(): snap_id++. Return snap_id-1.`

`get(index, snap_id): binary search in store[index] for rightmost entry with snap_id <= snap_id.`

Return that value.

WHY not store full array per snap: O(n * snaps) memory. Instead, only store changes.

WHY binary search: snap_ids in store[index] are monotonically increasing -> bisect_right-1.

Time: O(1) set, O(1) snap, O(log s) get where s = number of snaps. Space: O(n + total_sets).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
import bisect
class _1146_SnapshotArray:
    def __init__(self, length):
        self._snap_id = 0
        self._data = [[[0, 0]] for _ in range(length)]

    def set(self, index, val):
        if self._data[index][-1][0] == self._snap_id:
            self._data[index][-1][1] = val
        else:
            self._data[index].append([self._snap_id, val])

    def snap(self):
        sid = self._snap_id
        self._snap_id += 1
        return sid

    def get(self, index, snap_id):
        history = self._data[index]
        pos = bisect.bisect_right(history, [snap_id, float
('inf')]) - 1
```

```
        return history[pos][1]
```

PHASE 5 TEST & DEBUG

"Let me trace SnapshotArray(3): set(0,5),snap(),set(0,6),get(0,0),get(0,1)."

Each index stores [(snap_id, value)] list.

init: data=[[],[],[]], snap_id=0.

`set(0,5): data[0]=[(0,5)].`

`snap(): returns snap_id=0. snap_id=1.`

`set(0,6): data[0]=[(0,5),(1,6)].`

`get(0,0): bisect data[0] for snap_id=0. Find (0,5) -> return 5 ok`

`get(0,1): bisect data[0] for snap_id=1. Find (1,6) -> return 6 ok`

WHY sparse storage + bisect: only store changes, not full copy per snap. O(log(snaps)) per get.

PHASE 6 COMPLEXITY & FOLLOW-UP

Space O(changes): only store when values change. Time O(log s) per get.

Follow-up: if many sets with same value, deduplicate consecutive same-value entries.

Follow-up: persistent data structures — same concept generalised to any data structure.

MEDIUM | Binary Search | LeetCode #1482 | 19 of 21 | 19/21

Minimum Number of Days to Make m Bouquets

QUESTION

You are given an integer array bloomDay, an integer m and an integer k.

You want to make m bouquets. To make a bouquet, you need to use k adjacent flowers from the garden.

The garden consists of n flowers, the ith flower will bloom in the bloomDay[i] and then can be used in exactly one bouquet.

Return the minimum number of days you need to wait to be able to make m bouquets from the garden. If it is impossible to make m bouquets return -1.

Example 1

Input	`bloomDay = [1,10,3,10,2], m = 3, k = 1`
Output	`3`
Explanation	*Let us see what happened in the first three days. x means flower bloomed and _ means flower did not bloom in the garden.* *We need 3 bouquets each should contain 1 flower.* *After day 1: [x, _, _, _, _] // we can only make one bouquet.* *After day 2: [x, _, _, _, x] // we can only make two bouquets.* *After day 3: [x, _, x, _, x] // we can make 3 bouquets. The answer is 3.*

Example 2

Input	`bloomDay = [1,10,3,10,2], m = 3, k = 2`
Output	`-1`
Explanation	*We need 3 bouquets each has 2 flowers, that means we need 6 flowers. We only have 5 flowers so it is impossible to get the needed bouquets and we return -1.*

QUESTION

Example 3

Input	`bloomDay = [7,7,7,7,12,7,7], m = 2, k = 3`
Output	`12`
Explanation	*We need 2 bouquets each should have 3 flowers.* *Here is the garden after the 7 and 12 days:* *After day 7: [x, x, x, x, _, x, x]* *We can make one bouquet of the first three flowers that bloomed. We cannot make another bouquet from the last three flowers that bloomed because they are not adjacent.* *After day 12: [x, x, x, x, x, x, x]* *It is obvious that we can make two bouquets in different ways.*

CONSTRAINTS

- bloomDay.length == n
- 1 <= n <= 10^5
- 1 <= bloomDay[i] <= 10^9
- 1 <= m <= 10^6
- 1 <= k <= n

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

`bloomDay[i] = day flower i blooms. To make a bouquet need k adjacent flowers.`

Find minimum day when we can make m bouquets, or -1 if impossible. Right?"

"A few quick questions:

Need m*k flowers total in groups of k adjacent? Yes. If n < m*k return -1? Yes."

"Let me walk: bloomDay=[1,10,3,10,2], m=3, k=1 -> day 3: flowers at idx 0,2,4 bloomed -> 3 bouquets ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for day = 1 to the maximum bloom day:
    count the bouquets that can be made from
        flowers bloomed by that day
return the first day with at least m
```

TIME	SPACE
O(n * max(bloomDay))	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Tries every day. The number of bouquets only grows as days pass, so binary search on the day with an O(n) check is O(n log max).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Binary search on days; check if m bouquets can be made — O(n log n). Walk-through:"

`canMake(days): count consecutive bloomed flowers (bloomDay[i] <= days).`

Reset consecutive count when flower not yet bloomed.

When consecutive == k: bouquet made, count++; reset consecutive.

Return count >= m.

Binary search: lo=min(bloomDay), hi=max(bloomDay).

If canMake(mid): hi=mid. Else: lo=mid+1.

Return lo if canMake(lo) else -1.

WHY binary search: more days -> more flowers bloomed -> more bouquets -> monotone property.

WHY check if canMake(lo) at end: lo*=max(bloomDay) might not be feasible (m*k > n).

Time: O(n log(max_bloom)). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1482_MinDaysToMakeBouquets:
    def minDays(self, bloomDay, m, k):
        n = len(bloomDay)
        if n < m * k:
            return -1
        def can_make(day):
            bouquets = consecutive = 0
            for d in bloomDay:
                if d <= day:
                    consecutive += 1
                    if consecutive == k:
                        bouquets += 1
                        consecutive = 0
                else:
                    consecutive = 0
            return bouquets >= m
        lo, hi = 1, max(bloomDay)
        while lo < hi:
            mid = (lo + hi) // 2
            if can_make(mid): hi = mid
            else: lo = mid + 1
        return lo
```

PHASE 5 TEST & DEBUG

"Let me trace minDays([1,10,3,10,2], m=3, k=1)."

Binary search on day D: can we make m=3 bouquets by day D?

To make a bouquet need k=1 consecutive bloomed flower

PHASE 5 TEST & DEBUG

Day check(D): count consecutive bloomed (bloom[i]<=D) sequences of length k

lo=1, hi=10. mid=5: bloomed=[1,3,2] (<=5). Groups of 1: [1],[3],[2] -> 3>=3 -> hi=5

mid=3: bloomed=[1,3,2] wait: bloom=[1,10,3,10,2]. Day 3: [T,F,T,F,T]. Groups: {1},{3},{5} -> 3 bouquets -> hi=3

mid=2: [T,F,F,F,T]. Groups: {1},{5} -> 2<3 -> lo=3. lo==hi=3 -> return 3 ok

Greedy check: scan sorted positions, whenever k consecutive bloomed -> count bouquet. O(n) per check.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log D) where D = max(bloomDay)	**O(1)**

This is the classic 'binary search on the answer' pattern.

Follow-up: Capacity to Ship Packages (#1011) — same binary search template.

Follow-up: if we can choose which flowers to use (non-adjacent allowed), greedy suffices.

MEDIUM | Binary Search | LeetCode #1533 | 20 of 21 | 20/21

Find the Index of the Large Integer

QUESTION

We have an integer array arr, where all the integers in arr are equal except for one integer which is larger than the rest of the integers. You will not be given direct access to the array, instead, you will have an API ArrayReader which have the following functions:

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

One element is strictly larger than all others. Find its index using the compareSub API.

`compareSub(l,r,x,y) compares sum(arr[l..r]) with sum(arr[x..y]): returns 1,-1,0.`

At most 20 calls — so O(log n) calls are required. Right?"

"[7,7,7,7,10,7,7,7]: 3 compareSub calls suffice to find index 4 ok"

What should I return if the target is not found? Is the array always sorted?

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(1,_n-1_+_1):
    compareSub(0, 0, i, i) to see whether element
        i is bigger than element 0
    the larger one is the answer
```

TIME	SPACE
O(n) calls	O(1)

WHY IT IS SLOW

One API call per element. Compare the sums of two equal halves: the lighter half holds the smaller values, so the larger integer is in the heavier half; repeat: O(log n) calls.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Binary search: at each step, compare the left half against the right half of the range.

If left > right: large integer is in the left half -> search left.

If right > left: search right.

If equal: the large integer is the MIDDLE element (if odd-length range, middle was excluded).

Halve the range each step -> O(log n) calls."

PHASE 3 OPTIMIZE

Key insight: comparing left and right halves via the ArrayReader API narrows the large integer's location geometrically — at most O(log n) comparisons, far fewer than the O(n) naive linear scan.

PHASE 4 CLEAN CODE

Compare left half [lo..mid] with right half of equal size

```
class _1533_FindLargeInteger:
    def getIndex(self, reader) -> int:
        lo, hi = 0, reader.length() - 1
        while lo < hi:
            mid = (lo + hi) // 2
            left_end  = mid
            right_start = mid + 1
            right_end = right_start + (mid - lo)
            if right_end > hi:
                right_end = hi
            result = reader.compareSub(lo, left_end, right
_start, right_end)
            if result == 1:
                hi = left_end           # large int is in
left half
            elif result == -1:
                lo = right_start        # large int is in
right half
            else:
                return mid + 1          # equal sums -> la
rge int is the middle skipped element
        return lo
```

PHASE 5 TEST & DEBUG

"Let me trace getIndex(reader) where large integer at unknown position."

Binary search: lo=0, hi=len-1

Query compareSub(lo, lo+(hi-lo)//2, lo+(hi-lo)//2+1, hi)

If left half larger -> hi=mid; else lo=mid+1

Repeat until lo==hi -> return lo ok

WHY binary search: each compareSub eliminates half the candidates in O(log n) comparisons.

compareSub compares sum of digits in ranges; larger digit sum larger integer here.

Edge: single element -> return 0 immediately.

PHASE 6 COMPLEXITY & FOLLOW-UP

O(log n) compareSub calls. Well within the 20-call limit for n=5*10^5 (log(500000)~19).

The equal-sum case (odd-length range) means the middle element was the large one —

both halves have equal elements so the excluded middle is the outlier.

Follow-up: two large numbers? One comparison no longer definitively points to one half.

We'd need to track which half has MORE excess — more complex."

Space: O(n) — proportional to input or output size; O(1) extra if in-place.

IN YOUR APP | Expose | Editorial | Community

MEDIUM | Binary Search | LeetCode #1552 | 21 of 21 | 21/21

Magnetic Force Between Two Balls

QUESTION

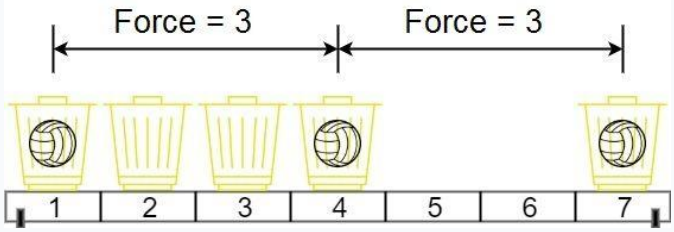
In the universe Earth C-137, Rick discovered a special form of magnetic force between two balls if they are put in his new invented basket. Rick has n empty baskets, the ith basket is at position[i], Morty has m balls and needs to distribute the balls into the baskets such that the minimum magnetic force between any two balls is maximum.

Rick stated that magnetic force between two different balls at positions x and y is |x - y|.

Given the integer array position and the integer m. Return the required force.

Example 1

Input	`position = [1,2,3,4,7], m = 3`
Output	`3`
Explanation	*Distributing the 3 balls into baskets 1, 4 and 7 will make the magnetic force between ball pairs [3, 3, 6]. The minimum magnetic force is 3. We cannot achieve a larger minimum magnetic force than 3.*



Example 2

Input	`position = [5,4,3,2,1,1000000000], m = 2`
Output	`999999999`
Explanation	*We can use baskets 1 and 1000000000.*

CONSTRAINTS

- n == position.length
- 2 <= n <= 10^5
- 1 <= position[i] <= 10^9
- All integers in position are distinct.
- 2 <= m <= position.length

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Place m balls in m of the n positions. Maximise the minimum distance between any two balls. Right?"

"A few quick questions:

Must sort positions first? Yes — we want to maximise spacing.

m >= 2 always? Yes per constraints."

"Let me walk: position=[1,2,3,4,7], m=3 -> place at 1,4,7: min dist=3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for way in ways:
    # to place m balls in the baskets
    take the smallest gap between any two
    best = max(best, candidate)
```

TIME	SPACE
O(C(n, m) * m)	O(m)

WHY IT IS SLOW

Tries every placement. Whether a minimum gap d is achievable is monotonic (greedy placement, each ball at least d after the last): binary search on d with an O(n) check is O(n log(range)).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the combinatorial search.

Binary search on the answer (minimum distance d).

Feasibility: can we place m balls with min distance >= d?

Greedy: place balls greedily — first ball at position[0], each subsequent ball at the next

position >= last + d.

Time: O(n log n + n log(max_dist)). Space: O(1)."

Key insight: "can we place m balls with min distance >= d?" is monotone decreasing — binary search on d in [1, max_pos - min_pos] finds the maximum valid minimum distance in O(n log max_pos).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1552_MagneticForceBetweenTwoBalls:
    def maxMinDist(self, position, m):
        position.sort()
        def can_place(min_dist):
            count, last = 1, position[0]
            for pos in position[1:]:
                if pos - last >= min_dist:
                    count += 1; last = pos
                    if count == m: return True
            return False
        lo, hi = 1, position[-1] - position[0]
        while lo < hi:
            mid = (lo + hi + 1) // 2
            if can_place(mid): lo = mid
            else: hi = mid - 1
```

```
        return lo
```

PHASE 5 TEST & DEBUG

"Let me trace maxDistance([1,2,3,4,7], m=3)."

Binary search on minimum distance D

lo=1, hi=(7-1)/(3-1)=3

Check D=2: place balls at 1, then >=3(2+2=3)->3, then >=5(3+2=5)->7? 7>=5 ok -> 3 balls placed -> valid -> lo=2

Check D=3: place at 1, then >=4(1+3=4)->4, then >=7(4+3=7)->7 ok -> valid -> lo=3

lo==hi=3 -> return 3 ok

WHY binary search: monotone — if D works, D-1 also works; find largest valid D.

Greedy placement: always place next ball at first position >= prev+D.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n log D) where D = max spread	**O(1)**

Maximize-the-minimum is the dual of Minimize-the-maximum — same binary search approach.

Follow-up: Split Array Largest Sum (#410) — same binary search template, different feasibility.

Follow-up: Aggressive Cows (classic USACO) — identical formulation.